กรณีทดสอบแบบเหตุการณ์ · อำพร OMS

ใช้ทดสอบด้วยตัวเองก่อนสอบ แต่ละกรณีบอกเหตุการณ์ ขั้นตอน ผลที่คาดหวัง และเส้นทางในโค้ดว่ามาจากโมดูลไหน เรียกจากไหน · อ้างอิงกรณีทดสอบ TC-01 ถึง TC-13 ตารางที่ 3.29 และเกณฑ์ AC ตารางที่ 3.11

ก่อนเริ่ม

1. เปิดระบบ: `cd soml` แล้ว `npm run dev` หน้าจออยู่ที่ `http://localhost:5173` หลังบ้าน `:4000` บริการพิมพ์ `:4100`
2. อยากเริ่มจากข้อมูลสะอาด: `npm run db:reset` (ล้างฐานใช้งานกลับเป็นสินค้า 783 รายการและผู้ใช้ตั้งต้น)
3. ชื่อผู้ใช้ทั้งสามบทบาทอยู่ใน `db/seed/seed-users.js`
4. มือถือใช้ `http://<IP เครื่อง>:5173` ใน Wi-Fi เดียวกัน

เส้นทางที่ทุกกรณีเดินผ่าน

หน้าจอ	frontend/src/pages/<ชื่อหน้า>/index.jsx
เรียก API	frontend/src/services/api.js (แนบโทเคน) · ข้อมูลสดใช้ services/events.js
ประตูหลังบ้าน	backend/app.js ประกาศทุกเส้นทาง → middleware/auth.js verifyToken → requireRole
ตรรกะ	backend/modules/<โมดูล>/index.js
ธุรกรรม	backend/config/db.js withTransaction() commit หรือ rollback ทั้งก้อน
หลัง commit	queue.broadcast() ผลักไปทุกจอ · print-client สั่งพิมพ์ · alert ตรวจแจ้งเตือน

รายการกรณีทดสอบ

รหัส	เหตุการณ์	ผู้ทดสอบ	อ้างอิง	หน้า
S-01	พนักงานขายเข้ากะเช้า	พนักงานขาย	UC-01 · FR-01 · TC-01 · AC-12	3
S-02	พนักงานขายพยายามเข้าหน้าผู้จัดการ	พนักงานขาย	UC-01 · FR-01 · NFR-01 · TC-01 · AC-01	4
S-03	ผู้จัดการตั้งจุดสั่งซื้อเพิ่มของปูน	ผู้จัดการ	UC-02 · FR-02 · TC-02 · AC-02	5
S-04	ของเข้าร้าน ผู้จัดการปรับสต็อก	ผู้จัดการ	UC-02 · FR-02 · NFR-04	6
S-05	ลูกค้าซื้อสามรายการ พนักงานคิดยอด	พนักงานขาย	UC-03 · FR-03 · TC-03 · AC-03	7
S-06	ลูกค้าโอนแล้ว พนักงานยืนยันรับชำระ	พนักงานขาย	UC-03 · UC-04 · FR-03 · TC-03 · TC-04 · TC-12	8
S-07	คลังเห็นออเดอร์เองโดยไม่ต้องรีเฟรช	พนักงานขาย + พนักงานคลัง	UC-05 · FR-04 · NFR-02 · TC-05 · AC-06	9
S-08	Wi-Fi ในโกดังหลุดแล้วกลับมา	พนักงานขาย + พนักงานคลัง	FR-04 · NFR-02 (แก้ 14 ต.ค. 2569)	10
S-09	คลังกดเริ่มจัดของ	พนักงานคลัง	UC-06 · FR-04 · ตารางที่ 3.23	11
S-10	ลูกค้ามารับของ คลังยืนยันจ่าย	พนักงานคลัง	UC-07 · FR-05 · FR-06 · TC-06	12
S-11	ลูกค้าเอาบิลเดิมกลับมารับของซ้ำ	พนักงานคลังสองคน	FR-06 · BR-02 · NFR-04 · TC-06 · AC-07 (ทดสอบหัวใจ)	13
S-12	ขายเกินของที่มี แล้วคลังจะจ่าย	ผู้จัดการ + พนักงานขาย + พนักงานคลัง	FR-06 · TC-07 · AC-08	14
S-13	ของใกล้หมด ผู้จัดการได้แจ้งเตือน	พนักงานคลัง + ผู้จัดการ	UC-11 · FR-08 · BR-05 · TC-09 · AC-10	15

S-14	ออเดอร์ค้างในคิวนานผิดปกติ	ผู้จัดการ	UC-11 · FR-08 · TC-10 · AC-13	16
S-15	ผู้จัดการดูยอดระหว่างวัน	ผู้จัดการ	UC-08 · UC-09 · FR-07 · TC-08 · AC-09	17
S-16	ยอดไม่ตรง ผู้จัดการตามหาว่าใครทำ	ผู้จัดการ	UC-10 · FR-09 · TC-11 · AC-11	18
S-17	พิมพ์ใบเสร็จภาษาไทยที่ร้าน	พนักงานขาย	FR-03 · NFR-05 · TC-12 · AC-14 · DF-03	19
S-18	ช่วงคนเยอะ ใช้งานพร้อมกันหลายเครื่อง	ทุกบทบาท (สคริปต์)	NFR-06 · NFR-04 · TC-13 · AC-15 · DF-01	20

คำสั่งรันเทสอัตโนมัติ (อาจารย์อาจขอให้รันให้ดู)

ต้องการ	คำสั่ง (รันในโฟลเดอร์ soml)
ทั้งหมด (unit + integration 134 ข้อ)	`npm test`
ไฟล์เดียว เช่น กันจ่ายซ้ำ	`npx jest backend/tests/integration/dispatch.test.js`
เฉพาะ unit	`npm run test:unit`
system test บนเบราว์เซอร์ (ต้องเปิดระบบก่อน)	`npm run test:system`
system test ข้อเดียว เช่น AC-07	`npx playwright test -g "AC-07"`
ดูรายงานล่าสุดของ playwright	`npx playwright show-report`

ตัวเลขที่อ้าง (134 ข้อ, 13 กรณี, 80 ms, 2.3 วินาที, 15,383 คำขอ) มาจากผลที่บันทึกในเล่มบทที่ 4 ถ้าทดสอบเองได้ค่าต่างไป ให้จดค่าจริงไว้ในช่องหมายเหตุ

S-01 พนักงานขายเข้ากะเช้า

UC-01 · FR-01 · TC-01 · AC-12

> เข้าวันใหม่ พนักงานขายเปิดเครื่องหน้าร้านแล้วเข้าสู่ระบบ ต้องไปหน้าขายทันที และไม่เห็นเมนูของผู้จัดการ

ผู้ทดสอบ: พนักงานขาย

เงื่อนไขก่อนทดสอบ

ระบบเปิดอยู่ มีบัญชีพนักงานขาย (ชื่อผู้ใช้ใน db/seed/seed-users.js)

ขั้นตอน

1. เปิด http://localhost:5173/login
2. กรอกรหัสผ่านผิดหนึ่งครั้ง แล้วกดเข้าสู่ระบบ
3. กรอกให้ถูก แล้วกดเข้าสู่ระบบ
4. ดูเมนูด้านซ้าย

ผลที่คาดหวัง

- รหัสผิดได้ข้อความแจ้ง ไม่เข้าระบบ
- รหัสถูกไปหน้าขายหน้าร้าน (/pos) เอง
- เมนูมีแค่ "ขายหน้าร้าน" ไม่มีสินค้า แดชบอร์ด หรือประวัติ

มาจากโมดูลไหน เรียกจากไหน

หน้าจอ	pages/login/index.jsx → services/session.jsx (login)
API	POST /api/auth/login
Backend	modules/auth/index.js → login() ตรวจรหัสด้วย bcrypt แล้วออก JWT อายุ 8 ชั่วโมง
ตาราง	อ่าน users · เขียน audit_logs (LOGIN)
หลังเข้า	session.jsx เรียก GET /api/auth/me → auth.me · เมนูมาจาก components/Layout.jsx ตาม role

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`frontend/src/services/session.jsx:24`	หน้าจอส่งชื่อผู้ใช้และรหัสผ่าน
`backend/app.js:34`	ประกาศเส้นทาง
`backend/modules/auth/index.js:28`	login() ตรวจรหัสด้วย bcrypt
`backend/modules/auth/index.js:43`	ออกโทเคน JWT
`backend/modules/auth/index.js:16`	อายุโทเคน 8 ชั่วโมง
`frontend/src/components/Layout.jsx:14`	เมนูของแต่ละบทบาท (roles)

> **ถ้าอาจารย์ถาม:** ทำไมไม่เก็บรหัสผ่านตรง ๆ → เก็บเป็นแฮช bcrypt ใน users.password_hash ถ้าฐานข้อมูลหลุด ก็ไม่ได้รหัสจริง

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-02 พนักงานขายพยายามเข้าหน้าผู้จัดการ

UC-01 · FR-01 · NFR-01 · TC-01 · AC-01

> พนักงานขายรู้ว่ามีหน้าจัดการสินค้า เลยลองพิมพ์ที่อยู่เข้าไปเอง และลองยิง API ตรง ๆ

ผู้ทดสอบ: พนักงานขาย

เงื่อนไขก่อนทดสอบ

เข้าสู่ระบบด้วยบัญชีพนักงานขายแล้ว

ขั้นตอน

1. พิมพ์ http://localhost:5173/products ในช่องที่อยู่
2. เปิด http://localhost:4000/api/products/1 ด้วยเบราว์เซอร์อีกแท็บ (ไม่มีโทเคน)
3. ใน Terminal รัน curl -i -X PUT http://localhost:4000/api/products/1 (ไม่แนบโทเคน)
4. ที่หน้าขาย กด F12 แท็บ Console พิมพ์ fetch('/api/products/1',{method:'PUT',headers:{Authorization:'Bearer '+localStorage.getItem('soml.token')}}).then(r=>r.status) (แนบโทเคนพนักงานขาย)

ผลที่คาดหวัง

- หน้าจอเด้งกลับไปหน้าขายของตัวเอง
- ไม่มีโทเคนได้ 401 "ต้องเข้าสู่ระบบก่อน"
- มีโทเคนพนักงานขายแต่เรียกเส้นทางผู้จัดการได้ 403 "บทบาทของคุณไม่มีสิทธิ์ทำรายการนี้"

มาจากโมดูลไหน เรียกจากไหน

หน้าจอ	App.jsx → components/RequireRole.jsx (roles ไม่ตรง → Navigate กลับหน้าแรกของบทบาท)
API	ทุกเส้นทางใน backend/app.js ผ่าน verifyToken แล้วตามด้วย requireRole(...)
Backend	middleware/auth.js → verifyToken() คืน 401 · requireRole() คืน 403

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`frontend/src/App.jsx:38`	หน้าสินค้าให้เฉพาะ manager
`frontend/src/components/RequireRole.jsx:10`	บทบาทไม่ตรง → เด้งกลับ
`backend/middleware/auth.js:23`	verifyToken() ไม่มีหรือโทเคนปลอม → 401
`backend/middleware/auth.js:42`	requireRole() บทบาทไม่ตรง → 403
`backend/app.js:58`	เส้นทางที่ลองยิง

> **ถ้าอาจารย์ถาม:** กันที่หน้าจออย่างเดียวพอไหม → ไม่พอ เพราะยิง API ตรงได้ จึงต้องกันที่เซิร์ฟเวอร์ทุกเส้นทาง (หน้าจอซ่อนเมนูแค่เพื่อความสะดวก)

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-03 ผู้จัดการตั้งจุดสั่งซื้อเพิ่มของปูน

UC-02 · FR-02 · TC-02 · AC-02

> ผู้จัดการอยากรู้ตัวก่อนของหมด จึงตั้งจุดสั่งซื้อเพิ่มของสินค้าตัวหนึ่ง

ผู้ทดสอบ: ผู้จัดการ

เงื่อนไขก่อนทดสอบ

เข้าสู่ระบบด้วยบัญชีผู้จัดการ

ขั้นตอน

1. เปิดเมนูจัดการข้อมูลสินค้า
2. ค้นสินค้าหนึ่งรายการ กดแก้ไข
3. ใส่จุดสั่งซื้อเพิ่ม เช่น 20 แล้วบันทึก
4. เปิดเมนูประวัติการทำรายการ

ผลที่คาดหวัง

- ค่าในตารางเปลี่ยนเป็น 20
- ประวัติมีแถว "UPDATE_PRODUCT" พร้อมชื่อผู้จัดการและเวลา

มาจากโมดูลไหน เรียกจากไหน

หน้าจอ	pages/products/index.jsx
API	PUT /api/products/:id (requireRole manager)
Backend	modules/product/index.js → update() ในธุรกรรม withTransaction (config/db.js)
ตาราง	แก้ products.reorder_point · เขียน audit_logs (UPDATE_PRODUCT)

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`frontend/src/pages/products/index.jsx:34`	หน้าจอส่งค่าที่แก้
`backend/app.js:58`	ประกาศเส้นทาง (manager)
`backend/modules/product/index.js:107`	update() ในธุรกรรม
`backend/modules/audit-log/index.js:38`	record() บันทึกประวัติ

> **ถ้าอาจารย์ถาม:** ทำไมใช้ธุรกรรมแม้แก้แถวเดียว → เพราะต้องเขียน products กับ audit_logs พร้อมกัน ถ้าอย่างหนึ่งพลาดต้องย้อนทั้งคู่

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-04 ของเข้าร้าน ผู้จัดการปรับสต็อก

UC-02 · FR-02 · NFR-04

> รถส่งของมาส่งสินค้า ผู้จัดการเพิ่มยอดคงเหลือพร้อมเหตุผล

ผู้ทดสอบ: ผู้จัดการ

เงื่อนไขก่อนทดสอบ

เข้าสู่ระบบด้วยบัญชีผู้จัดการ

ขั้นตอน

1. หน้าจัดการสินค้า กดปรับสต็อกของสินค้าหนึ่งรายการ
2. ใส่จำนวน +10 เหตุผล "รับสินค้าเข้า" แล้วบันทึก
3. ลองใส่ -9999 (มากกว่าที่มี)

ผลที่คาดหวัง

- ยอดคงเหลือเพิ่ม 10
- ใส่ติดลบเกินที่มี ได้ข้อความ "ยอดคงเหลือจะติดลบ" (409) ยอดไม่เปลี่ยน

มาจากโมดูลไหน เรียกจากไหน

หน้าจอ	pages/products/index.jsx
API	PATCH /api/products/:id/stock
Backend	modules/product/index.js → adjustStock() ล็อกแถวด้วย SELECT ... FOR UPDATE แล้วหลัง commit เรียก alert.checkStock()
ตาราง	แก้ products.stock_qty · เขียน stock_movements และ audit_logs (ADJUST_STOCK)

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`frontend/src/pages/products/index.jsx:40`	หน้าจอส่งจำนวนและเหตุผล
`backend/modules/product/index.js:144`	adjustStock()
`backend/modules/product/index.js:156`	ล็อกแถวสินค้า
`backend/modules/product/index.js:159`	กันยอดติดลบ
`backend/modules/product/index.js:163`	บันทึกการเคลื่อนไหว
`backend/modules/product/index.js:173`	ตรวจแจ้งเตือนหลัง commit

> **ถ้าอาจารย์ถาม:** stock_movements มีไว้ทำไม → เก็บทุกครั้งที่สต็อกเปลี่ยน พร้อมยอดหลังเปลี่ยนและสาเหตุ ทำให้ตรวจย้อนได้ว่าสต็อกหายไปเพราะอะไร

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-05 ลูกค้าซื้อสามรายการ พนักงานคิดยอด

UC-03 · FR-03 · TC-03 · AC-03

> ลูกค้ามาซื้อเหล็ก สี และท่อ พนักงานเลือกสินค้าลงรายการ ระบบคิดยอดให้ และแสดง QR ให้ลูกค้าสแกน

ผู้ทดสอบ: พนักงานขาย

เงื่อนไขก่อนทดสอบ

เข้าสู่ระบบด้วยบัญชีพนักงานขาย ตั้ง PROMPTPAY_ID ใน .env แล้ว

ขั้นตอน

1. ค้นและเพิ่มสินค้า 3 รายการ
2. เปลี่ยนจำนวนของรายการแรกเป็น 2
3. คิดเลขเองบนกระดาษ (ราคา × จำนวน รวมกัน)
4. กด "แสดงรหัส QR เพื่อรับชำระ"
5. ลองสแกน QR ด้วยแอปธนาคาร (ไม่ต้องโอนจริง) ดูยอดที่ขึ้น

ผลที่คาดหวัง

- ยอดบนจอตรงกับที่คิดเอง
- ยอดบนหน้า QR และยอดที่แอปธนาคารอ่านจาก QR ตรงกัน

มาจากโมดูลไหน เรียกจากไหน

หน้าจอ	pages/pos/index.jsx (ใช้ components/Money.jsx แสดงเงิน · ไลบรารี qrcode วาด QR)
API	GET /api/products (ค้นสินค้า) · POST /api/orders/quote (คิดยอด)
Backend	modules/order/index.js → quote() → parseItems() → priceItems() อ่านราคาจากฐานข้อมูล → lib/promptpay.js buildPayload()
ตาราง	อ่าน products อย่างเดียว ยังไม่บันทึกอะไร

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`frontend/src/pages/pos/index.jsx:52`	ค้นสินค้า
`frontend/src/pages/pos/index.jsx:62`	ขอให้หลังบ้านคิดยอด
`backend/app.js:44`	ประกาศเส้นทาง (sales, manager)
`backend/modules/order/index.js:93`	quote()
`backend/modules/order/index.js:57`	priceItems() ราคาจากฐานข้อมูล
`backend/lib/promptpay.js:56`	สร้างข้อความ QR พร้อมเพย์

> **ถ้าอาจารย์ถาม:** ทำไมไม่ใช้ราคาที่หน้าจอส่งมา → หน้าจอแก้ได้ จึงคิดราคาจากฐานข้อมูลที่เซิร์ฟเวอร์เท่านั้น (หน้าจอส่งแค่รหัสสินค้ากับจำนวน)

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-06 ลูกค้าโอนแล้ว พนักงานยืนยันรับชำระ

UC-03 · UC-04 · FR-03 · TC-03 · TC-04 · TC-12

> ลูกค้าโชว์สลิป พนักงานดูแอปธนาคารว่าเงินเข้า แล้วกดยืนยัน ระบบออกเลขที่ พิมพ์ใบเสร็จ และส่งคำสั่งซื้อไปคลัง

ผู้ทดสอบ: พนักงานขาย

เงื่อนไขก่อนทดสอบ

ต่อจาก S-05 บริการพิมพ์เปิดอยู่ (พอร์ต 4100)

ขั้นตอน

1. ตรวจแอปธนาคารว่าเงินเข้าจริง
2. กด "ยืนยันรับชำระเงิน"
3. ดูเลขที่คำสั่งซื้อที่ได้ และใบเสร็จที่ออกจากเครื่องพิมพ์
4. ขายต่ออีกใบ ดูว่าเลขที่ต่อเนื่องไม่ซ้ำ

ผลที่คาดหวัง

- ได้เลขที่ใหม่ ไม่ซ้ำกับใบก่อน
- ใบเสร็จภาษาไทยออกภายใน 5 วินาที คอลัมน์ตรง
- ถ้าปิดบริการพิมพ์ไว้ การขายยังสำเร็จ (แค่แจ้งว่าพิมพ์ไม่ได้)

มาจากโมดูลไหน เรียกจากไหน

หน้าจอ	pages/pos/index.jsx → confirmPayment()
API	POST /api/orders
Backend	modules/order/index.js → create() ในธุรกรรม: nextOrderNo() ล็อกเลขล่าสุดของวันด้วย FOR UPDATE → priceItems() → บันทึก · ชนกันลองใหม่สูงสุด 12 ครั้ง (DF-01)
หลัง commit	modules/print-client → printReceipt() → print-service/server.js POST /print/receipt (ESC/POS) · queue.broadcast("order.created")
ตาราง	เขียน orders, order_items (เก็บราคา ณ เวลาขาย), audit_logs (CREATE_ORDER)

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`frontend/src/pages/pos/index.jsx:96`	กดยืนยันรับชำระ
`frontend/src/pages/pos/index.jsx:99`	ส่งคำสั่งซื้อ
`backend/modules/order/index.js:119`	create()
`backend/modules/order/index.js:81`	nextOrderNo() ล็อกเลขล่าสุดด้วย FOR UPDATE
`backend/modules/order/index.js:170`	ชนกันแล้วลองใหม่ (DF-01)
`backend/modules/order/index.js:177`	สั่งพิมพ์หลัง commit
`backend/modules/order/index.js:188`	ผลักไปจอคลัง
`print-service/server.js:21`	บริการพิมพ์รับงาน

> **ถ้าอาจารย์ถาม:** ทำไมสั่งพิมพ์หลัง commit → ถ้าพิมพ์ล่มการขายต้องยังสำเร็จ และคิวต้องไม่เห็นคำสั่งซื้อที่ยังไม่บันทึกจริง

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-07 คลังเห็นออเดอร์เองโดยไม่ต้องรีเฟรช

UC-05 · FR-04 · NFR-02 · TC-05 · AC-06

ไฮไลต์ของระบบ: ขายที่หน้าร้าน แล้วรายการขึ้นที่จอคลังเอง คลังเริ่มจัดของได้ก่อนลูกค้าเดินไปถึง

ผู้ทดสอบ: พนักงานขาย + พนักงานคลัง

เงื่อนไขก่อนทดสอบ

เปิดสองหน้าจอ: คอมเข้าบัญชีพนักงานขาย และมือถือเข้าบัญชีพนักงานคลังที่หน้าคิว (http://<IP เครื่อง>:5173)

ขั้นตอน

1. ที่มือถือเปิดหน้าคิวค้างไว้ ห้ามกดรีเฟรช
2. ที่คอมขายสินค้าหนึ่งใบ แล้วกดยืนยันรับชำระ
3. จับเวลาตั้งแต่กดจนรายการขึ้นที่มือถือ

ผลที่คาดหวัง

- รายการขึ้นที่มือถือเองภายใน 3 วินาที (วัดได้สูงสุด 80 ms)
- หน้าคิวมีป้ายว่ารับข้อมูลสดอยู่

มาจากโมดูลไหน เรียกจากไหน

หน้าจอ	pages/queue/index.jsx → subscribe() ใน services/events.js เปิด WebSocket ที่ /ws (ถ้าเปิดไม่ได้ใน 3 วินาที ถอยไปใช้ SSE ที่ /api/events)
API	GET /api/queue ทุกครั้งที่ได้เหตุการณ์ order.*
Backend	modules/queue/index.js → attach() (WebSocket) · events() (SSE) · broadcast() · list()
ตาราง	อ่าน orders, order_items, products

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`frontend/src/pages/queue/index.jsx:28`	load() ดึงคิว
`frontend/src/pages/queue/index.jsx:32`	ฟังเหตุการณ์ order.*
`frontend/src/services/events.js:19`	subscribe()
`frontend/src/services/events.js:29`	เปิด WebSocket
`frontend/src/services/events.js:41`	ทางสำรอง SSE
`backend/modules/queue/index.js:38`	broadcast() ส่งทุกจอ
`backend/modules/queue/index.js:49`	attach() รับ WebSocket ที่ /ws
`backend/modules/queue/index.js:92`	list() ส่งรายการคิว

ถ้าอาจารย์ถาม: ทำไมไม่ให้จอวนถามทุก 1–2 วินาที → เปลืองและช้ากว่า ใช้ push แทน (ADR-02) และมี SSE สำรองเพราะอุปกรณ์เครือข่ายบางรุ่นไม่ยอมให้ใช้ WebSocket

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-08 Wi-Fi ในโกดังหลุดแล้วกลับมา

FR-04 · NFR-02 (แก้ 14 ต.ค. 2569)

> สัญญาณในโกดังหลุดชั่วครู่ ระหว่างนั้นหน้าร้านขายของไปหนึ่งใบ พอสัญญาณกลับ คิวต้องไม่ตกหล่น

ผู้ทดสอบ: พนักงานขาย + พนักงานคลัง

เงื่อนไขก่อนทดสอบ

เหมือน S-07

ขั้นตอน

1. ที่มือถือปิด Wi-Fi
2. ที่คอมขายหนึ่งใบ
3. เปิด Wi-Fi มือถือกลับ รอ 2–3 วินาที ห้ามกดรีเฟรช

ผลที่คาดหวัง

- ใบที่ขายตอนหลุดขึ้นมาในคิวเอง

มาจากโมดูลไหน เรียกจากไหน

หน้าจอ	services/events.js: เมื่อต่อได้ (ws.onopen / sse.onopen) ส่งเหตุการณ์ "connected" · ต่อใหม่ทุก 2 วินาทีเมื่อหลุด
หน้าจอ	pages/queue/index.jsx: event === "connected" → load() ดึงคิวใหม่ทั้งหมด
API	GET /api/queue

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`frontend/src/services/events.js:17`	ต่อใหม่ทุก 2 วินาที
`frontend/src/services/events.js:32`	ต่อ WebSocket ได้ → ส่ง connected
`frontend/src/services/events.js:42`	ต่อ SSE ได้ → ส่ง connected
`frontend/src/pages/queue/index.jsx:32`	ได้ connected → load() ใหม่ทั้งหมด

> **ถ้าอาจารย์ถาม:** ทำไมต้องโหลดใหม่ทั้งหมด → เหตุการณ์ที่เกิดระหว่างหลุดไม่ถูกส่งซ้ำ การดึงคิวทั้งก้อนจากฐานข้อมูลจึงเป็นวิธีที่ถูกต้องที่สุด

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-09 คลังกดเริ่มจัดของ

UC-06 · FR-04 · ตารางที่ 3.23

> พนักงานคลังรับงานใบหนึ่ง กด "เริ่มจัด" ให้คนอื่นเห็นว่ามีคนทำแล้ว

ผู้ทดสอบ: พนักงานคลัง

เงื่อนไขก่อนทดสอบ

มีคำสั่งซื้อสถานะรอจ่ายในคิว

ขั้นตอน

1. กดเริ่มจัดที่ใบหนึ่ง
2. ดูอีกจอว่าสถานะเปลี่ยนตาม
3. กด "ยกเลิกการจัด" ให้กลับเป็นรอจ่าย
4. ลองให้พนักงานคลังยกเลิกคำสั่งซื้อ (ไม่มีปุ่ม) แล้วให้ผู้จัดการยกเลิก

ผลที่คาดหวัง

- สถานะเปลี่ยนเป็นกำลังจัด ทุกจอเห็นพร้อมกัน
- ย้อนกลับเป็นรอจ่ายได้
- ยกเลิกคำสั่งซื้อได้เฉพาะผู้จัดการ

มาจากโมดูลไหน เรียกจากไหน

หน้าจอ	pages/queue/index.jsx
API	PATCH /api/orders/:id/status
Backend	modules/queue/index.js → updateStatus() ตรวจ TRANSITIONS (สถานะที่ไปต่อได้) และ WHO_CAN (ยกเลิกเฉพาะ manager) ล็อกแถวด้วย FOR UPDATE · หลัง commit broadcast("order.status")
ตาราง	แก้ orders.status · เขียน audit_logs (UPDATE_STATUS / CANCEL_ORDER)

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`frontend/src/pages/queue/index.jsx:39`	กดเริ่มจัด
`backend/modules/queue/index.js:24`	สถานะที่ไปต่อได้
`backend/modules/queue/index.js:31`	ใครเปลี่ยนเป็นอะไรได้
`backend/modules/queue/index.js:109`	updateStatus()
`backend/modules/queue/index.js:132`	ผลักสถานะใหม่

> **ถ้าอาจารย์ถาม:** เปลี่ยนเป็น "ส่งมอบแล้ว" ผ่านเส้นทางนี้ได้ไหม → ไม่ได้ TRANSITIONS ตั้งใจไม่ใส่ delivered ต้องผ่านการยืนยันจ่ายที่ตัดสต็อกเท่านั้น

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-10 ลูกค้ามารับของ คลังยืนยันจ่าย

UC-07 · FR-05 · FR-06 · TC-06

> ลูกค้ายื่นเลขที่คำสั่งซื้อ คลังเปิดใบ จัดของครบ แล้วกดยืนยันจ่าย ระบบตัดสต็อกให้

ผู้ทดสอบ: พนักงานคลัง

เงื่อนไขก่อนทดสอบ

มีคำสั่งซื้อรอจ่ายหรือกำลังจัด จดยอดคงเหลือของสินค้าในใบไว้ก่อน

ขั้นตอน

1. แตะใบในคิวเพื่อเปิดหน้ายืนยันจ่าย
2. กด "ยืนยันจ่ายสินค้า" แล้วกดยืนยันอีกครั้ง
3. ดูคิวว่าใบหายไป
4. ผู้จัดการเปิดหน้าสินค้าดูยอดคงเหลือ

ผลที่คาดหวัง

- ต้องกดสองครั้ง (กันกดพลาด)
- ใบหายจากคิวทุกจอ
- สต็อกลดเท่าจำนวนที่จ่ายพอดี

มาจากโมดูลไหน เรียกจากไหน

หน้าจอ	pages/dispatch/index.jsx (arming = ขั้นยืนยันซ้ำ)
API	GET /api/orders/:id · POST /api/orders/:id/dispatch
Backend	modules/dispatch/index.js → confirm() → withTransaction → modules/stock-deduction/index.js → dispatch()
ในธุรกรรม	1) UPDATE orders … WHERE status IN (รอจ่าย, กำลังจัด) 2) UPDATE products SET stock_qty = stock_qty – ? WHERE stock_qty >= ? 3) บันทึก stock_movements 4) audit_logs (CONFIRM_DISPATCH)
หลัง commit	broadcast("order.dispatched") · alert.checkStock()

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`frontend/src/pages/dispatch/index.jsx:31`	กดยืนยันจ่าย (หลังยืนยันซ้ำ)
`backend/app.js:52`	ประกาศเส้นทาง (warehouse, manager)
`backend/modules/dispatch/index.js:16`	confirm() เปิดธุรกรรม
`backend/modules/stock-deduction/index.js:25`	dispatch() ตัดสต็อก
`backend/modules/stock-deduction/index.js:32`	เปลี่ยนสถานะแบบมีเงื่อนไข (กันจ่ายซ้ำ)
`backend/modules/stock-deduction/index.js:51`	หักสต็อกแบบมีเงื่อนไข (กันติดลบ)
`backend/modules/stock-deduction/index.js:61`	บันทึกการเคลื่อนไหว
`backend/config/db.js:41`	withTransaction() commit/rollback

> **ถ้าอาจารย์ถาม:** ตัดสต็อกตอนขายหรือตอนจ่าย → ตอนจ่าย เพราะของออกจากคลังจริงตอนนั้น ใบที่ยกเลิกก่อนจ่ายจึงไม่ต้องคืนสต็อก

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-11 ลูกค้าเอาบิลเดิมกลับมารับของซ้ำ

FR-06 · BR-02 · NFR-04 · TC-06 · AC-07 (ทดสอบหัวใจ)

> ปัญหาเดิมของร้าน (EV-06): มีคนเอาบิลเก่ามารับของซ้ำ หรือพนักงานสองคนกดจ่ายใบเดียวกันพร้อมกัน

ผู้ทดสอบ: พนักงานคลังสองคน

เงื่อนไขก่อนทดสอบ

มีคำสั่งซื้อรอจ่ายหนึ่งใบ เปิดหน้ายืนยันจ่ายของใบนั้นไว้สองเครื่อง

ขั้นตอน

1. เครื่องแรกกดยืนยันจ่ายจนสำเร็จ
2. เครื่องที่สองกดยืนยันจ่ายใบเดิม
3. แบบชนกันจริงระดับเสี้ยววินาที ให้รัน: npx jest backend/tests/integration/dispatch.test.js

ผลที่คาดหวัง

- เครื่องที่สองได้ข้อความ "ถูกจ่ายหรือยกเลิกไปแล้ว" (409 ALREADY_DISPATCHED)
- สต็อกถูกหักครั้งเดียว
- เทสอัตโนมัติผ่าน: สองคำขอพร้อมกันได้ 200 หนึ่งครั้ง 409 หนึ่งครั้ง

มาจากโมดูลไหน เรียกจากไหน

Backend	modules/stock-deduction/index.js บรรทัด 31–38: UPDATE แบบมีเงื่อนไขสถานะ ถ้า affectedRows = 0 โยน ALREADY_DISPATCHED
Backend	modules/dispatch/index.js แปลงเป็น HTTP 409
เทส	backend/tests/integration/dispatch.test.js บรรทัด 73–81 (Promise.all ยิงพร้อมกัน)

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`backend/modules/stock-deduction/index.js:37`	affectedRows = 0 → ปฏิเสธ
`backend/modules/dispatch/index.js:25`	แปลงเป็น HTTP 409
`backend/tests/integration/dispatch.test.js:76`	เทสยิงพร้อมกันสองคำขอ

> **ถ้าอาจารย์ถาม:** ทำไมกดด้วยมือทดสอบไม่พอ → สองคนกดให้ชนกันในเสี้ยววินาทีทำไม่ได้จริง ต้องใช้สคริปต์ยิงพร้อมกันเพื่อให้ได้ผลซ้ำทุกครั้ง

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-12 ขายเกินของที่มี แล้วคลังจะจ่าย

FR-06 · TC-07 · AC-08

> ยอดในระบบเหลือน้อย แต่หน้าร้านขายไปมากกว่านั้น พอคลังกดจ่าย ระบบต้องไม่ยอมให้สต็อกติดลบ

ผู้ทดสอบ: ผู้จัดการ + พนักงานขาย + พนักงานคลัง

เงื่อนไขก่อนทดสอบ

ผู้จัดการปรับสต็อกสินค้าหนึ่งรายการให้เหลือ 1 (S-04)

ขั้นตอน

1. พนักงานขายขายสินค้านั้น 2 ชิ้นในใบเดียวกับสินค้าอื่น (ระบบยอมให้ขาย เพราะตัดสต็อกตอนจ่าย)
2. คลังกดยืนยันจ่ายใบนั้น
3. ดูสต็อกของทุกรายการในใบ และสถานะใบ

ผลที่คาดหวัง

- ได้ข้อความ "สต็อก ... ไม่พอสำหรับจำนวน 2" (409 INSUFFICIENT_STOCK)
- สต็อกทุกรายการในใบไม่เปลี่ยนเลย สถานะใบยังรอจ่าย ไม่มี stock_movements ใหม่

มาจากโมดูลไหน เรียกจากไหน

Backend	modules/stock-deduction/index.js บรรทัด 49–56: WHERE stock_qty >= ? ถ้าไม่ผ่านโยน INSUFFICIENT_STOCK
ธุรกรรม	config/db.js → withTransaction() rollback ทั้งใบ รวมรายการที่หักไปแล้วก่อนหน้า

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`backend/modules/stock-deduction/index.js:55`	สต็อกไม่พอ → โยนข้อผิดพลาด
`backend/config/db.js:49`	ย้อนทั้งใบ

> **ถ้าอาจารย์ถาม:** TC-06 กับ TC-07 ต่างกันยังไง → TC-06 ล็อกที่ระดับคำสั่งซื้อ (กันจ่ายซ้ำ) TC-07 ล็อกที่ระดับสินค้า (กันติดลบ) ต้องผ่านทั้งคู่

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-13 ของใกล้หมด ผู้จัดการได้แจ้งเตือน

UC-11 · FR-08 · BR-05 · TC-09 · AC-10

> ปัญหาเดิม (EV-09): สั่งของจากความรู้สึก พอสต็อกต่ำกว่าจุดสั่งซื้อเพิ่ม ระบบต้องเตือนเอง

ผู้ทดสอบ: พนักงานคลัง + ผู้จัดการ

เงื่อนไขก่อนทดสอบ

สินค้าหนึ่งรายการ stock 12 จุดสั่งซื้อเพิ่ม 10 (ตั้งด้วย S-03, S-04)

ขั้นตอน

1. ขายสินค้านั้น 5 ชิ้นแล้วคลังจ่าย (เหลือ 7 ต่ำกว่า 10)
2. ดูกระดิ่งแจ้งเตือนและหน้าการแจ้งเตือน
3. ขายและจ่ายอีก 1 ชิ้นโดยยังไม่กดอ่านแล้ว

ผลที่คาดหวัง

- มีแจ้งเตือน "... เหลือ 7 ... ต่ำกว่าจุดสั่งซื้อเพิ่ม (10)" ขึ้นทันที ตัวเลขกระดิ่งเพิ่มเอง
- จ่ายครั้งที่สองไม่เกิดแจ้งเตือนซ้ำ

มาจากโมดูลไหน เรียกจากไหน

Backend	modules/dispatch/index.js หลัง commit → modules/alert/index.js → checkStock() → createOnce() (ไม่สร้างซ้ำถ้ายังมีอันที่ยังไม่อ่าน) → broadcast("notification.created")
หน้าจอ	pages/notifications/index.jsx · services/unread.js (ตัวเลขกระดิ่ง) · API GET /api/notifications · กดอ่านแล้ว PATCH /api/notifications/:id/read (เฉพาะผู้จัดการ)
ตาราง	เขียน notifications · audit_logs (CREATE_NOTIFICATION ผู้กระทำคือระบบ user_id = NULL)

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`backend/modules/dispatch/index.js:33`	ตรวจหลังจ่าย
`backend/modules/alert/index.js:42`	checkStock()
`backend/modules/alert/index.js:22`	createOnce() ไม่สร้างซ้ำ
`backend/modules/alert/index.js:34`	ผลักแจ้งเตือน
`frontend/src/pages/notifications/index.jsx:34`	ปุ่มอ่านแล้วเฉพาะผู้จัดการ
`frontend/src/services/unread.js:19`	ตัวเลขกระดิ่ง

> **ถ้าอาจารย์ถาม:** ทำไมตรวจหลัง commit ไม่ใช่ในธุรกรรม → การแจ้งเตือนพลาดต้องไม่ทำให้การจ่ายของล้ม

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-14 ออเดอร์ค้างในคิวนานผิดปกติ

UC-11 · FR-08 · TC-10 · AC-13

> ช่วงคนเยอะ มีใบหนึ่งค้างไม่มีใครจัด ระบบต้องเตือนผู้จัดการเอง

ผู้ทดสอบ: ผู้จัดการ

เงื่อนไขก่อนทดสอบ

แก้ .env ให้ QUEUE_DELAY_THRESHOLD_MINUTES=1 แล้วเปิดระบบใหม่ (ค่าจริงคือ 30 นาที อย่าลืมแก้คืน)

ขั้นตอน

1. ขายหนึ่งใบแล้วปล่อยไว้ในคิว ไม่จัด
2. รอประมาณ 2 นาที
3. ดูหน้าการแจ้งเตือนของผู้จัดการ

ผลที่คาดหวัง

- มีแจ้งเตือน "... รอจ่ายสินค้ามา ... นาที" และไม่ซ้ำทุกนาที

มาจากโมดูลไหน เรียกจากไหน

Backend	backend/server.js เรียก alert.start() ตอนเปิดระบบ → setInterval ทุก 60 วินาที → checkQueueDelay() หาใบที่ paid_at เกินเกณฑ์ → createOnce()
ตาราง	อ่าน orders · เขียน notifications, audit_logs

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`backend/server.js:17`	เริ่มตรวจตอนเปิดระบบ
`backend/modules/alert/index.js:19`	ตรวจทุก 60 วินาที
`backend/modules/alert/index.js:18`	เกณฑ์จาก .env
`backend/modules/alert/index.js:63`	checkQueueDelay()

> **ถ้าอาจารย์ถาม:** ใครเป็นผู้กระทำ → ระบบเอง (ผู้กระทำอัตโนมัติในแผนภาพ use case) ไม่ต้องมีคนกด

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-15 ผู้จัดการดูยอดระหว่างวัน

UC-08 · UC-09 · FR-07 · TC-08 · AC-09

> ปัญหาเดิม (EV-08): ต้องรอสรุปยอดตอนปิดร้าน ตอนนี้ต้องเห็นได้ทันที

ผู้ทดสอบ: ผู้จัดการ

เงื่อนไขก่อนทดสอบ

มีการขายและจ่ายในวันนี้แล้วหลายใบ

ขั้นตอน

1. เปิดแดชบอร์ดปฏิบัติการ จดยอดขายวันนี้และจำนวนคิวค้าง
2. รวมยอดเองจากใบที่ขายวันนี้ (หรือรัน SQL: SELECT SUM(total_amount) FROM orders WHERE DATE(paid_at)=CURDATE() AND status<>'cancelled')
3. ขายเพิ่มอีกหนึ่งใบ ดูว่าแดชบอร์ดเปลี่ยนเอง
4. เปิดแดชบอร์ดระดับบริหาร เปลี่ยนช่วงวัน

ผลที่คาดหวัง

- ตัวเลขบนแดชบอร์ดตรงกับที่รวมเอง
- ขายเพิ่มแล้วตัวเลขเปลี่ยนเองไม่ต้องรีเฟรช

มาจากโมดูลไหน เรียกจากไหน

หน้าจอ	pages/dashboard-ops/index.jsx (โหลดใหม่ทุกเหตุการณ์) · pages/dashboard-exec/index.jsx
API	GET /api/dashboard/ops · GET /api/dashboard/exec?days=...
Backend	modules/dashboard/index.js → ops() · exec()
ตาราง	อ่าน orders, order_items, products, notifications

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`frontend/src/pages/dashboard-ops/index.jsx:35`	หน้าจอเรียกข้อมูล
`backend/modules/dashboard/index.js:15`	ไม่นับใบที่ยกเลิก
`backend/modules/dashboard/index.js:20`	ops() ยอดวันนี้ คิว ของใกล้หมด
`backend/modules/dashboard/index.js:50`	exec() ระดับบริหาร

> **ถ้าอาจารย์ถาม:** ตัวเลขแดชบอร์ดเก็บไว้ในตารางไหม → ไม่เก็บ คำนวณจากตารางจริงทุกครั้งที่เรียก ตัวเลขจึงตรงกับข้อมูลเสมอ

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-16 ยอดไม่ตรง ผู้จัดการตามหาว่าใครทำ

UC-10 · FR-09 · TC-11 · AC-11

> ปัญหาเดิม (EV-02): ข้อมูลกระจัดกระจาย ตรวจย้อนหลังยาก ตอนนี้ต้องค้นได้ว่าใครทำอะไรเมื่อไร

ผู้ทดสอบ: ผู้จัดการ

เงื่อนไขก่อนทดสอบ

ทำ S-03 ถึง S-10 มาแล้ว

ขั้นตอน

1. เปิดประวัติการทำรายการ
2. กรองช่วงวันที่วันนี้
3. กรองประเภท "ยืนยันจ่ายสินค้า"
4. ดูแถวที่ได้

ผลที่คาดหวัง

- แสดงเฉพาะแถวที่ตรงเงื่อนไข
- ทุกแถวมีผู้กระทำ เวลา สิ่งที่ถูกกระทำ และไอพี

มาจากโมดูลไหน เรียกจากไหน

หน้าจอ	pages/audit-logs/index.jsx
API	GET /api/audit-logs?...
Backend	modules/audit-log/index.js → list() · ทุกโมดูลเรียก record() ภายในธุรกรรมของตัวเอง
ตาราง	อ่าน audit_logs (ACTIONS 10 ชนิด เช่น LOGIN, CREATE_ORDER, CONFIRM_DISPATCH)

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`frontend/src/pages/audit-logs/index.jsx:28`	หน้าจอส่งตัวกรอง
`backend/modules/audit-log/index.js:17`	ประเภทการกระทำ 10 ชนิด
`backend/modules/audit-log/index.js:38`	record() ทุกโมดูลเรียก
`backend/modules/audit-log/index.js:52`	list() ค้นตามเงื่อนไข

> **ถ้าอาจารย์ถาม:** ทำไมเขียน audit ในธุรกรรมเดียวกับงาน → ถ้างานถูกย้อน ประวัติก็ย้อนด้วย ไม่มีประวัติของสิ่งที่ไม่เกิดขึ้นจริง

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-17 พิมพ์ใบเสร็จภาษาไทยที่ร้าน

FR-03 · NFR-05 · TC-12 · AC-14 · DF-03

ที่ร้าน (Windows) เครื่องพิมพ์ความร้อนต่อ USB และแชร์ชื่อ AmpornOMS-Receipt ขายจากมือถือแล้วใบเสร็จต้องออกเร็ว

ผู้ทดสอบ: พนักงานขาย

เงื่อนไขก่อนทดสอบ

ติดตั้งตามภาคผนวก จ เปิดระบบด้วย start.bat

ขั้นตอน

1. ขายจากมือถือ
2. กดยืนยันรับชำระพร้อมจับเวลา
3. หยุดเวลาเมื่อเครื่องพิมพ์ตัดกระดาษ
4. ตรวจภาษาไทยและคอลัมน์

ผลที่คาดหวัง

- เสร็จภายใน 5 วินาที (วัดได้ประมาณ 2.3 วินาที)
- ภาษาไทยไม่เพี้ยน คอลัมน์ตรง 48 ตัวอักษร

มาจากโมดูลไหน เรียกจากไหน

Backend	modules/print-client/index.js → printReceipt() (ไม่โยนข้อผิดพลาด หมดเวลา 5 วินาที)
Print Service	print-service/server.js POST /print/receipt → lib (ESC/POS ภาษาไทย รหัส มอก. 620) → เครื่องพิมพ์ที่แชร์
เทส	backend/tests/unit/escpos.test.js · print-client.test.js

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`backend/modules/print-client/index.js:21`	printReceipt() ไม่โยนข้อผิดพลาด
`backend/modules/print-client/index.js:15`	หมดเวลา 5 วินาที
`print-service/server.js:21`	แปลงเป็น ESC/POS แล้วส่งเครื่องพิมพ์

ถ้าอาจารย์ถาม: ทำไมพิมพ์ผ่านเครื่องพิมพ์ที่แชร์ → ส่ง USB ตรงบน Windows ไม่ได้ (DF-03) จึงย้อนกลับไปแก้วิธีพิมพ์

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____

S-18 ช่วงคนเยอะ ใช้งานพร้อมกันหลายเครื่อง

NFR-06 · NFR-04 · TC-13 · AC-15 · DF-01

> ช่วงเร่งด่วนมีหลายเครื่องขายและจ่ายพร้อมกัน ระบบต้องไม่ล้มและเลขที่ต้องไม่ซ้ำ

ผู้ทดสอบ: ทุกบทบาท (สคริปต์)

เงื่อนไขก่อนทดสอบ

ระบบเปิดครบทั้งสี่หน่วย

ขั้นตอน

1. รัน npm run evidence (TC-05 วัดความหน่วง และ TC-13 ยิง 10 เซสชัน 20 วินาที)
2. ดูผลใน evidence/tc13.json

ผลที่คาดหวัง

- ไม่มีคำขอล้มเหลว (รอบที่บันทึกในเล่ม 15,383 คำขอ ล้มเหลว 0)
- เลขที่คำสั่งซื้อไม่ซ้ำ

มาจากโมดูลไหน เรียกจากไหน

สคริปต์	evidence/measure-tc05.js · evidence/measure-tc13.js · evidence/measure-tc04.js
Backend	modules/order/index.js → nextOrderNo() FOR UPDATE + ลองใหม่แบบถอยหลังทวีคูณ (retryDelayMs) เมื่อชนกัน

บรรทัดโค้ด (ใน VS Code กด Cmd+P แล้วพิมพ์ ไฟล์:บรรทัด เช่น stock-deduction/index.js:32 · ไฟล์อยู่ในโฟลเดอร์ soml)

`backend/modules/order/index.js:22`	ลองใหม่สูงสุด 12 ครั้ง
`backend/modules/order/index.js:25`	ถอยหลังแบบทวีคูณ
`evidence/measure-tc13.js:130`	สคริปต์ TC-13 เขียนผล

> **ถ้าอาจารย์ถาม:** DF-01 คืออะไร → รอบแรกของ TC-13 มีคำขอล้ม 2 ครั้งเพราะธุรกรรมชนกันตอนออกเลขที่ จึงเพิ่มการลองใหม่อัตโนมัติ แล้ววัดซ้ำจนไม่ล้มเลย

☐ ผ่าน ☐ ไม่ผ่าน วันที่ทดสอบ _____ หมายเหตุ _____